

Epicordia — Development Plan for Antigravity

Phased build plan for the offline Flutter board/notes/task app — each phase is a self-contained directive with a hard quality gate before the next phase begins.

How to use this document

Feed Antigravity **one phase at a time**, in order. Each phase below is written so you can copy it directly as a directive. Every phase ends with a **Definition of Done** and a **Gate Check** — since you're running agents autonomously rather than reviewing line-by-line, the Gate Check is what replaces your manual review: Antigravity must run it and report a pass before you move to the next phase's directive. If a Gate Check fails, the directive is "fix the failing criteria in Phase N before doing anything else" — never patch forward into the next phase.

Global rules to include at the top of every phase directive:

1. Do not begin work on anything outside this phase's scope.
 2. Write automated tests for everything built in this phase (unit tests for logic/schema, widget tests for UI) before declaring the phase done.
 3. Commit to git at logical checkpoints within the phase, with clear messages — never one giant commit per phase.
 4. At the end of the phase, run the full existing test suite (not just this phase's new tests) and confirm nothing earlier broke.
 5. Produce a short written summary of what was built, what was tested, and any deviations from the directive, before stopping.
 6. If something in this phase is ambiguous or conflicts with earlier work, stop and flag it rather than guessing silently.
-

Phase 0 — Project Foundation

Goal: A clean, empty, correctly configured Flutter project that runs on Android, iOS, Web/Desktop targets, with the core architecture wired but no features yet.

Tasks:

- ☐ Initialize Flutter project with folder structure separating `data/` (Drift schema, repositories), `domain/` (models, business logic), `presentation/` (screens, widgets), `core/` (theming, utils, routing).

- ☐ Add and configure: Drift (local DB), Riverpod (state management), `go_router` or equivalent for navigation, `home_widget` (widgets, stubbed for now), a chosen infinite-canvas package.
- ☐ Set up responsive layout scaffolding (breakpoints for phone / tablet / desktop) with a placeholder "Hello world" screen proving it adapts across three sample screen sizes.
- ☐ Set up app theming skeleton (light/dark, one accent color) per the UI direction in the spec.
- ☐ Set up CI-equivalent local scripts: `flutter analyze` and `flutter test` runnable as one command.
- ☐ Confirm the app builds and launches on at least: Android emulator, iOS simulator (or documented reason if unavailable), and one desktop target (Windows/macOS/Linux, whichever is the dev machine), plus web if targeted.

Definition of Done:

- App launches with zero errors on all targeted platforms.
- `flutter analyze` returns no errors.
- Folder structure matches the separation above.
- A placeholder screen visibly reflows correctly at phone/tablet/desktop widths.

Gate Check (must pass before Phase 1):

- ☐ Clean checkout + `flutter run` succeeds on each target platform without manual fixes.
 - ☐ Test suite (even if minimal) runs and passes.
 - ☐ Written confirmation of which platforms were actually verified vs. assumed.
-

Phase 1 — Data Layer (Drift Schema)

Goal: The complete local database schema for the whole app exists, is tested, and is provably correct — before any UI is built on top of it. This is the highest-risk phase to get wrong, since dependencies, recurrence, and calendar sync all rely on this schema later.

Tasks:

- ☐ Design and implement Drift tables for: `Board` (id, title, parentBoardId nullable, defaultViewMode, kanbanEnabled boolean — opt-in per board, milestoneDate nullable, createdAt, modifiedAt), `Pin` (id, boardId, type, x, y, width, height, zIndex, rotation, colorTag, content — polymorphic per pin type, createdAt, modifiedAt), `Task` (id, pinId nullable if it's a standalone task, title, notes, dueDate, scheduledDate nullable — separate "work on this" date distinct from dueDate, priority, status/kanbanStage nullable — only

meaningful when the parent board has `kanbanEnabled`, `boardId`, `recurrenceRule` nullable, `calendarEventId` nullable), `TaskDependency` (`taskId`, `dependsOnTaskId`), `Connector` (`fromPinId`, `toPinId`, `boardId`), plus supporting tables for attachments/files (path references, not blobs).

- ☐ Implement cycle-detection logic for `TaskDependency` (a task cannot directly or transitively depend on itself) as a pure, unit-testable function — this must exist before any UI lets a user create a dependency.
- ☐ Implement reactive Drift queries/streams for: all boards, pins for a given board, tasks due today, tasks by board, notes across all boards, tasks blocked vs. unblocked.
- ☐ Write a data migration strategy stub (even if v1 has only one schema version, the migration path must exist so v2 doesn't require a rewrite).
- ☐ Write unit tests covering: CRUD for every table, cycle detection (positive and negative cases), cascading delete behavior (deleting a board deletes its pins; deleting a parent board with nested boards — decide and test the intended behavior explicitly), and query correctness for the reactive streams above.

Definition of Done:

- All tables exist and match the field lists above (or a documented, justified deviation).
- Cycle detection is proven correct via tests with at least: no dependency, valid chain, direct cycle, indirect/transitive cycle.
- Nested board deletion behavior is explicitly decided, implemented, and tested (not left as accidental behavior).
- 100% of new schema/query code has unit test coverage.

Gate Check (must pass before Phase 2):

- ☐ Full test suite passes, including all Phase 1 tests.
 - ☐ A short data-model diagram or table list is produced so you can visually confirm the schema before UI work begins.
 - ☐ Confirm explicitly: can a task exist without ever being on a board (for quick-capture/Inbox)? This must be answered and tested, since it affects the Notes/Tasks tabs in Phase 2.
-

Phase 2 — Core Simple Views (No Canvas Yet)

Goal: The app is already fully usable as a plain notes + to-do app, proving Pillar 4 ("canvas is optional") works before the harder canvas engine is built. This deliberately sequences the *safer* views first.

Tasks:

- ☐ Build the **Today dashboard** as the app's landing screen: tasks due today/overdue, grouped by board, with quick-complete and quick-edit.
- ☐ Build the **global Notes tab**: flat, searchable list of all notes across all boards, inline edit.
- ☐ Build the **global Tasks tab**: flat, filterable/sortable list of all tasks, with kanban-stage and priority filters, and visible blocked/unblocked state from Phase 1's dependency data.
- ☐ Build **Quick Capture**: a floating action button + modal sheet that creates a note or task into a default "Inbox" board without requiring board selection first.
- ☐ Build **Zen/Focus capture mode** as the default rendering of Quick Capture: only a text field and a save action visible — no board picker, no tags, no color, no kanban stage. All of that can be added later by editing the item from its Inbox location, but must not be presented at the moment of capture.
- ☐ Ensure the Today dashboard and task editors clearly distinguish **due date** vs. **scheduled/start date** as two separate, separately-editable fields (not one field reused for both meanings).
- ☐ Build basic **List view** rendering for a single board (task checklist + note cards, no positions needed) — this is the simplified per-board mode from the spec, built before Canvas view.
- ☐ Wire local notifications for due tasks/reminders (no calendar sync yet — just in-app due-date based).
- ☐ Confirm full responsiveness of every screen built in this phase across phone/tablet/desktop.

Definition of Done:

- A user can open the app, land on Today, capture a note or task in under 3 taps, see it in the Notes or Tasks tab, and mark tasks complete — entirely without ever touching a canvas.
- List view for a board correctly reflects the same underlying data as the DB (no divergent state).
- Notifications fire correctly for a due task in a manual test.

Gate Check (must pass before Phase 3):

- ☐ Full test suite passes (unit + widget tests for every screen above).
- ☐ Manual verification checklist completed and reported: create note → appears in Notes tab; create task with due date → appears in Today and Tasks tab; complete task → disappears/updates correctly in all views; app fully navigable on all three screen-size classes.

- ☐ Explicit confirmation that the app is shippable as a "notes + tasks only" app at this checkpoint — this is your first real usability milestone, independent of the canvas ever being finished.
-

Phase 3 — Boards & Canvas Engine

Goal: The infinite canvas is added as an additional view mode on top of the already-working data and List view from Phase 2 — never replacing it.

Tasks:

- ☐ Integrate the chosen infinite-canvas package; implement pan/zoom on all platforms (touch gestures on mobile, scroll+drag on desktop).
- ☐ Implement drag-and-drop placement of pins onto the canvas from a pin tray, using (x,y) from the Phase 1 schema.
- ☐ Implement pin dragging/repositioning, multi-select (marquee + shift-click), snapping/alignment guides.
- ☐ Implement nested boards: dragging a board pin onto another board files it inside; breadcrumb navigation bar; double-tap/click to enter a nested board.
- ☐ Implement the **Canvas / List / Focus toggle** per board, confirming switching modes never loses or duplicates data (this is the critical regression risk from Phase 2 — test explicitly).
- ☐ Implement Focus view (kanban-style columns by status, using Phase 1's stage field).
- ☐ Build the pin editors for the core types only in this phase: Note, Task card, Image, Link — remaining pin types (drawing, audio, connectors, frames) are explicitly deferred to Phase 6.

Definition of Done:

- A board can be created, filled with Note/Task/Image/Link pins via drag-drop, nested inside another board, and switched between Canvas/List/Focus without any data loss or divergence.
- Performance is acceptable with at least 100 pins on one board (no visible jank on mid-range hardware) — test and report actual numbers.

Gate Check (must pass before Phase 4):

- ☐ Full test suite passes.
- ☐ Explicit regression check: re-run every Phase 2 manual verification item to confirm the canvas addition didn't break the simple-view experience.
- ☐ Report performance numbers for the 100-pin test.

- ☐ Confirm nested board depth works to at least 3 levels deep with breadcrumb navigation correct at each level.
-

Phase 4 — Project Management Depth

Goal: Kanban stages, milestones, and task dependencies become fully usable end-to-end, building on schema already proven in Phase 1.

Tasks:

- ☐ Implement the per-board **kanban opt-in toggle**: when off (the default for new boards), Focus view shows a plain checklist grouped by scheduled/due date instead of status columns, and tasks on that board have no kanban stage at all. When a board owner enables kanban, Focus view switches to To Do/Doing/Done columns for that board only.
- ☐ Wire kanban stage changes (drag between Focus-view columns, or manual field edit) to persist and reflect everywhere (Today, Tasks tab, board List/Canvas view) — only for boards with kanban enabled.
- ☐ Implement board-level milestones/deadlines with a visible indicator on the board (canvas and list) and in a milestones-aware view (e.g. shown in Calendar in Phase 5, or a simple upcoming-milestones list now).
- ☐ Implement the dependency UI: marking Task B as depending on Task A, visually showing "blocked" state (greyed/locked card on canvas, badge in Focus/kanban and Tasks tab), and preventing completion of a blocked task (or warning clearly if you decide to allow it — decide explicitly and document).
- ☐ Implement recurring tasks: recurrence rule editor (daily/weekly/monthly/custom), and generation of future task instances from a recurring task.
- ☐ Re-run cycle-detection from Phase 1 live in the UI: attempting to create a circular dependency must be blocked with a clear message, not silently allowed.

Definition of Done:

- A user can build a small project with 5+ tasks, set 2 of them as dependent, move stages, set a milestone date, and set one task to recur weekly — and all of this is reflected consistently across Canvas, List, Focus, Today, and Tasks tab.

Gate Check (must pass before Phase 5):

- ☐ Full test suite passes.
- ☐ Manual scenario above executed and reported step by step.
- ☐ Confirm attempting a circular dependency is blocked in the actual UI, not just at the unit-test level.

Phase 5 — Calendar Sync & Notifications

Goal: Two-way sync with the device's native calendar, integrated with recurrence from Phase 4, plus full notification coverage.

Tasks:

- ☐ Implement device calendar read access (permissions flow per platform) and display of relevant device events inside the app (e.g. on Today/Calendar view).
- ☐ Implement push of app tasks/deadlines to the device calendar as calendar events.
- ☐ Implement conflict handling: define and implement what happens when the same item changes on both sides between syncs (e.g. last-write-wins by timestamp, or a merge prompt) — document the chosen strategy explicitly.
- ☐ Map in-app recurrence rules to calendar recurrence rules (RRULE) so a repeating task and its calendar representation stay as one logical item, not duplicating on every sync.
- ☐ Build the **Calendar view** screen (month/week grid) combining app tasks and synced device events.
- ☐ Extend local notifications to cover recurring task instances and synced calendar items correctly (no duplicate or missing notifications).

Definition of Done:

- A task created in-app with a due date appears on the device's native calendar app, and an event created on the device calendar (in the synced calendar) appears in-app, without duplication after repeated syncs.
- A recurring task correctly produces a recurring calendar entry, not N separate one-off entries.

Gate Check (must pass before Phase 6):

- ☐ Full test suite passes.
 - ☐ Manual two-way sync test executed on at least one real device/emulator per platform, with before/after screenshots or description reported.
 - ☐ Explicit confirmation of the conflict-resolution rule chosen, with a test demonstrating it.
-

Phase 6 — Remaining Pin Types & Rich Media

Goal: Complete the full pin set deferred from Phase 3: drawing/handwriting, audio, connectors, frames, files, color swatches, headings.

Tasks:

- ☐ Implement the Drawing/Sketch pin: freehand canvas, stored as vector strokes, resizable after creation.
- ☐ Implement the Handwriting pin with stylus pressure support where the platform provides it; on-device handwriting-to-text is optional/stretch — implement only if time allows and mark clearly if deferred further.
- ☐ Implement Audio/voice memo pin: record, waveform preview, playback.
- ☐ Implement File/Document preview pin, Color swatch pin, Heading/Divider pin, Checklist (lightweight) pin.
- ☐ Implement Connector/Arrow pins between any two pins on a canvas.
- ☐ Implement Group/Frame pin: a resizable container where dragging the frame moves everything inside it.
- ☐ Update the pin tray and List/Focus renderers so every new pin type displays sensibly in all three view modes (a drawing shown as a thumbnail in List view, for example).

Definition of Done:

- Every pin type from the original spec list exists, can be created, edited, deleted, and correctly renders in Canvas, List, and (where applicable) Focus view.

Gate Check (must pass before Phase 7):

- ☐ Full test suite passes.
 - ☐ A checklist confirming each pin type individually was created and round-tripped (create → close app or navigate away → reopen → data intact).
-

Phase 7 — Security & Data Portability

Goal: App lock and export/import, the two features that make "no login, no cloud" actually livable.

Tasks:

- ☐ Implement optional PIN and biometric app lock (platform biometric APIs), with a clear fallback if biometrics are unavailable.
- ☐ Implement full workspace export: a single file (zip of DB + local asset files: images, audio, drawings, attachments).
- ☐ Implement import: restoring from an exported file, including a safe conflict path if importing into a device that already has data (e.g. merge vs. replace, chosen explicitly and documented).

- ☐ Implement **automatic scheduled local backups**: a background task that writes a timestamped backup (same format as manual export) to local storage on a regular interval (e.g. daily) and prunes old backups beyond a retention count, with a settings screen showing last backup time and a manual "restore from backup" picker.
- ☐ Test the full export-on-device-A → import-on-device-B loop end to end, including all pin types, dependencies, and recurrence data.

Definition of Done:

- App lock correctly gates access and cannot be trivially bypassed by backgrounding/relaunching the app.
- An exported file, imported on a second install (or after a full uninstall/reinstall), restores 100% of boards, pins, tasks, dependencies, recurrence rules, and file attachments.

Gate Check (must pass before Phase 8):

- ☐ Full test suite passes.
 - ☐ Manual export/import round-trip test executed and reported, ideally across two different devices/emulators.
 - ☐ App lock manually tested: lock engages correctly, wrong PIN/biometric fails correctly, correct one unlocks.
-

Phase 8 — Cross-Platform Polish & Responsive QA

Goal: Everything built in Phases 0–7 is verified and refined specifically for phone, tablet, and desktop as equally first-class experiences, per the original responsive requirement.

Tasks:

- ☐ Full UI pass on phone: verify all screens, gestures (drag-drop, pinch-zoom, long-press context menus) work correctly with touch.
- ☐ Full UI pass on tablet: verify layout takes advantage of extra space (side navigation, wider canvas, split views where sensible) rather than just stretching phone layouts.
- ☐ Full UI pass on desktop: verify mouse/keyboard interactions (scroll-zoom, click-drag, right-click menus, resizable panels) work correctly and the app doesn't feel like a stretched mobile app.
- ☐ Apply the visual design direction from the spec consistently: canvas dot-grid background, card shadows/rounded corners, single accent color, drag-lift micro-interaction, board-entry zoom transition, meaningful empty states.
- ☐ Accessibility pass: text scaling, color contrast, basic screen-reader labeling on core flows.

☐ Performance pass across the whole app: cold start time, canvas performance with a large board (200+ pins), search speed with a large dataset (1000+ notes/tasks) — record actual numbers.

Definition of Done:

- Every screen has been manually walked through on all three device classes with issues logged and fixed, not just "should work."
- Recorded performance numbers meet reasonable targets you sign off on (e.g. cold start under a few seconds, smooth 60fps-ish canvas interaction at typical board sizes).

Gate Check (must pass before Phase 9):

- ☐ Full test suite passes.
 - ☐ A per-platform QA report is produced (phone/tablet/desktop) listing what was tested and its result.
 - ☐ Performance numbers reported and explicitly accepted or flagged for further optimization.
-

Phase 9 — Release Preparation

Goal: The app is packaged and ready to install/distribute as a finished v1.

Tasks:

- ☐ App icons, splash screen, store metadata (even if not publishing to stores yet, prepare it).
- ☐ Final full regression pass: execute every Gate Check checklist from Phases 0–8 once more in sequence against the final build.
- ☐ Build release binaries/packages for each target platform.
- ☐ Write a short internal changelog/README describing what v1 includes, matching this plan, and any deviations made along the way.

Definition of Done:

- Installable release builds exist for every targeted platform, and the full regression pass across all prior phases is green.

Gate Check (final):

- ☐ All prior Gate Checks re-confirmed passing on the final build, not just at the time each phase was completed.

- ☐ You personally do a final hands-on walkthrough before considering v1 complete — this is the one point in the plan worth reviewing yourself directly, even though phases were built autonomously.
-

Notes on sequencing rationale

- **Phase 1 before any UI** — because dependencies, recurrence, and calendar sync all hinge on the schema being right; fixing schema mistakes after UI is built is far more expensive.
- **Phase 2 (simple views) before Phase 3 (canvas)** — deliberately proves the "canvas is optional" pillar works and gives you a usable app checkpoint early, rather than betting the whole first milestone on the hardest engineering piece (the canvas).
- **Phase 4 (PM depth) before Phase 5 (calendar)** — dependencies/recurrence need to be solid before you also map them onto external calendar recurrence rules.
- **Phase 6 (remaining pins) deliberately late** — these are additive and lower-risk; nothing else in the plan depends on drawing/audio pins existing.
- **Phase 8 (responsive polish) after everything, not per-phase** — earlier phases should build responsively from the start (per your requirement), but a dedicated final pass catches integration issues that only show up once every feature coexists.